

**OVERVIEW** · FastAPI learning notes

# FastAPI Learning Journey: Day-by-Day Guide

What each learning day covers, the key ideas, and where to find the code and notes

**Learning objectives**

After working through these notes you should be able to:

- ✓ See the whole learning path at a glance and how each day builds on the previous ones.
- ✓ Know the key concepts and the most important code pattern of every day.
- ✓ Find the right file to open: original code, commented code, or the detailed day notes PDF.

## 1. How to use this repository

Every day folder contains the same three resources. Start with this overview to pick a day, read the day's notes PDF to learn the concepts, then open the commented code to see them in your own program.

| File                                | What it is                                                                   |
|-------------------------------------|------------------------------------------------------------------------------|
| <code>dayN/main.py</code>           | the original code I wrote while learning                                     |
| <code>dayN/main_commented.py</code> | the same code with educational comments – same behavior, runnable            |
| <code>dayN/dayN_notes.pdf</code>    | detailed notes: objectives, explanations, examples, common mistakes, summary |

Run any day from the project root with `uv run fastapi dev dayN/main.py` and open `http://127.0.0.1:8000/docs`. Only the days that have material in the repository are listed (there is no day 1 or day 9 folder).

## 2. The learning path

| Day | Theme                                                 | Main tools                                                                                               |
|-----|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| 2   | Validating data in the URL                            | <code>Path</code> , <code>Query</code> , <code>Annotated</code>                                          |
| 3   | Pydantic models for query parameters and bodies       | <code>BaseModel</code> , <code>Field</code> , <code>Literal</code> , <code>HttpUrl</code>                |
| 4-5 | Shaping responses, errors and updates                 | <code>response_model</code> , <code>status</code> , <code>HTTPException</code> , <code>model_copy</code> |
| 6   | Dependency injection                                  | <code>Depends</code> , class dependencies                                                                |
| 7   | Dependencies with setup and cleanup                   | <code>yield</code> , <code>try/finally</code>                                                            |
| 8   | Automated testing                                     | <code>TestClient</code> , <code>pytest</code> , <code>assert</code>                                      |
| 10  | Real database with <code>SQLModel</code> + PostgreSQL | <code>Session</code> , <code>select</code> , <code>.env</code> , <code>URL.create</code>                 |

How the days build on each other

```

Day 2    validate parameters in the URL
|
Day 3    validate with Pydantic models
|
Day 4-5  control responses, errors and updates
|
Day 6    share logic with Depends
|
Day 7    dependencies with setup + cleanup (yield)
|
Day 8    test what you built
|
Day 10   database app: uses models (3, 4-5), PATCH (5), get_session (6, 7)

```

### 3. Day 2 - Path & query parameter validation

**Folder:** `day2/` · **Notes:** `day2_notes.pdf` (4 pages)

**What it covers:** the two places data lives in a URL. A path parameter (`/items/5`) is part of the path and always required. A query parameter (`?q=hello`) comes after `?` and becomes optional when it has a default. `Path()` adds numeric rules (`gt`, `ge`, `lt`, `le`) and `Query()` adds string rules (`min_length`, `max_length`, `pattern`, `alias`).

Key code

```

@app.get("/items/{item_id}")
async def get_item_id(item_id: Annotated[int, Path(gt=0, le=1000)],
                      q: Annotated[str | None, Query(min_length=3, max_length=50)] = None):
    return {"id": item_id, "q": q}

```

#### Remember

Invalid input never reaches your function: FastAPI answers with **422** and an error that names the field (`loc`), the problem (`msg`) and the rule (`ctx`).

### 4. Day 3 - Pydantic models for query parameters & bodies

**Folder:** `day3/` · **Notes:** `day3_notes.pdf` (3 pages)

**What it covers:** grouping several query parameters into one model (`FilterParams`) read with `Annotated[FilterParams, Query()]`; `Field()` for defaults and rules; `Literal` for fixed choices; `extra="forbid"` to reject unknown parameters. For request bodies: a nested model (`Image` inside `Item`), `HttpUrl` validation, and `set[str]` removing duplicates.

| Parameter looks like...                                   | FastAPI reads it from |
|-----------------------------------------------------------|-----------------------|
| name in the route path                                    | the path              |
| simple type, not in the path                              | the query string      |
| a Pydantic model                                          | the JSON body         |
| a model wrapped in <code>Annotated[Model, Query()]</code> | the query string      |

### 5. Days 4-5 - Responses, status codes, errors & updates

**Folder:** `day4-5/` · **Notes:** `day4-5_notes.pdf` (6 pages)

**Day 4:** a return type or `response_model` validates, filters and documents the response - used to keep a password out of the output with separate `UserIn` / `UserOut` models. Status codes (`status.HTTP_...`) and path operation configuration (`tags`, `summary`, `description`,

`response_description, deprecated)`.

**Day 5:** raising `HTTPException` (404 with a `detail`), a custom exception with `@app.exception_handler`, and body updates: PUT replaces, PATCH updates part. The partial-update recipe:

Key code

```
new_item = item.model_dump(exclude_unset=True)    # only fields the client sent
final_item = Item(**items[item_id]).model_copy(update=new_item)
items[item_id] = jsonable_encoder(final_item)
```

#### In your code

The custom handler returns **401**; 404 is the right code for “not found”. Explained in the comments and notes; the original code is unchanged.

## 6. Day 6 - Dependencies

**Folder:** `day6/` · **Notes:** `day6_notes.pdf` (4 pages) · also `day6_screen_recording.zip`

**What it covers:** dependency injection – an endpoint declares what it needs with `Depends(...)` and FastAPI provides it per request. A shared parameter function reused through an alias, sub-dependencies, and classes as dependencies (callables whose `__init__` parameters FastAPI reads), which give real attributes and editor autocomplete instead of a dict.

Key code

```
CommonsDep = Annotated[dict, Depends(common_parameters)]

@app.get("/items/")
async def read_items(common: CommonsDep):
    return common
```

#### Common mistake

Pass the function, don't call it: `Depends(common_parameters)`, not `Depends(common_parameters())`.

## 7. Day 7 - Dependencies with yield

**Folder:** `day7/` · **Notes:** `day7_notes.pdf` (3 pages)

**What it covers:** a dependency that uses `yield` has setup code (before), an injected value, and cleanup code (after, in `finally`). Chained dependencies open from the innermost outwards and close in reverse order.

Verified output of GET `/test/`

```
A open
B open
endpoint
B close
A close
```

This is exactly how Day 10 gives every request its own database session (`get_session`).

## 8. Day 8 - Testing

**Folder:** `day8/` · **Notes:** `day8_notes.pdf` (5 pages) · tests in `test_main.py`

**What it covers:** `TestClient(app)` sends fake HTTP requests to the app without a server; pytest runs every `test_*` function; assertions check the status code and the JSON body, for success **and** error cases.

Key code

```
response = client.get("/items/0")
assert response.status_code == 422      # Path(ge=1) rejects 0
```

Run with `uv run pytest day8 -v` (all tests pass).

## 9. Day 10 - SQL databases with SQLAlchemy + PostgreSQL

**Folder:** `day10/` · **Notes:** `day10_notes.pdf` (7 pages, includes the quiz review)

**What it covers:** `SQLModel` = `SQLAlchemy` + `Pydantic`; one engine per app, one session per request. Credentials come from `.env` and the connection URL is built with `URL.create`. One model per job (`Hero`, `HeroCreate`, `HeroUpdate`, `HeroPublic`), and full CRUD:

| Endpoint                              | Key idea                                                                                                                                   |
|---------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>POST /heroes/</code>            | <code>model_validate</code> → <code>add</code> → <code>commit</code> → <code>refresh</code> ; response filtered by <code>HeroPublic</code> |
| <code>GET /heroes/</code>             | <code>session.exec(select(Hero).offset(...).limit(...)).all()</code>                                                                       |
| <code>GET /heroes/{hero_id}</code>    | <code>session.get(Hero, hero_id)</code> → 404 if <code>None</code>                                                                         |
| <code>PATCH /heroes/{hero_id}</code>  | <code>model_dump(exclude_unset=True)</code> + <code>sqlmodel_update</code>                                                                 |
| <code>DELETE /heroes/{hero_id}</code> | <code>session.delete</code> + <code>commit</code>                                                                                          |

### Remember

Pydantic checks the shape (422), your code raises 404, and uncaught database errors become 500. Setup: copy `.env.example` to `.env` and create the `heroes_db` database.

## 10. Revision summary

- Day 2-3: validate everything that comes in – parameters with `Path/Query`, groups and bodies with models.
- Day 4-5: control what goes out – response models, status codes, errors, partial updates.
- Day 6-7: share logic and resources with dependencies, with `yield` for cleanup.
- Day 8: prove it works with `TestClient` and pytest.
- Day 10: put it all together on a real PostgreSQL database with `SQLModel`.